

---

Process Management

# Process Scheduling

---

Lecture 3 (**Live** Version)

# Overview

## ■ Concurrent Execution

## ■ Process Scheduling

- ❑ Definition
- ❑ Process behavior
- ❑ Processing environment
- ❑ Criteria for good scheduling
- ❑ Procedure of process scheduling

## ■ Scheduling Algorithms

- ❑ For interactive System
- ❑ For real-time systems

# Concurrent Execution

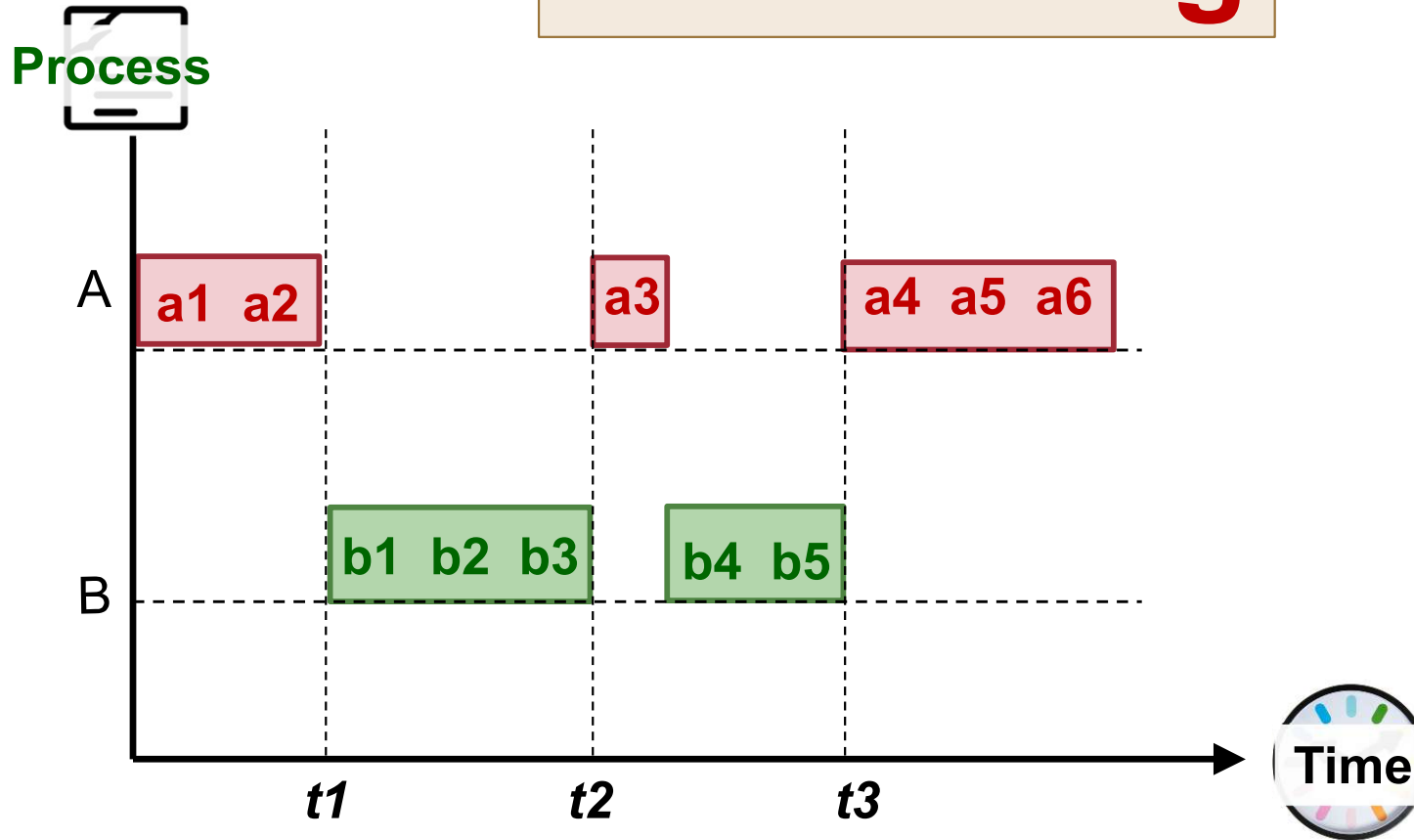
- **Concurrent processes:**

- ❑ Logical concept to cover multitasked processes
- ❑ Could be **virtual parallelism**
- ❑ Could be **physical parallelism**

- You can assume the two forms of parallelisms are not distinguished in the following discussion

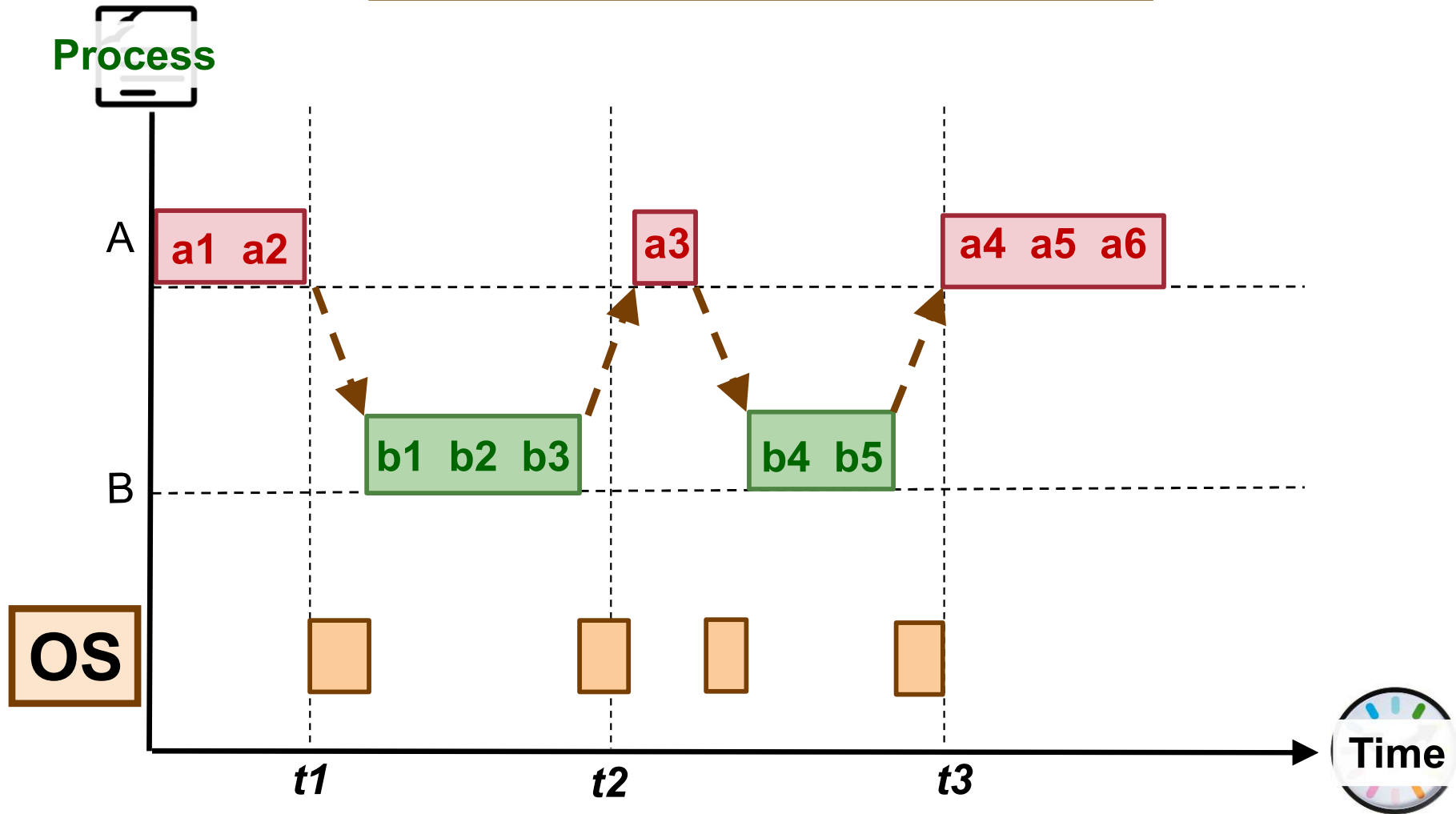
# Concurrency Example (Simplistic)

## Timeslicing



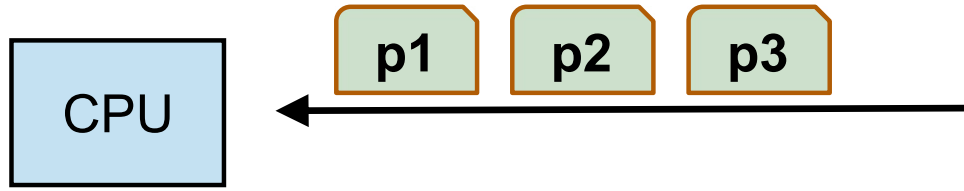
# Interleaved Execution (context switch)

## Context Switch

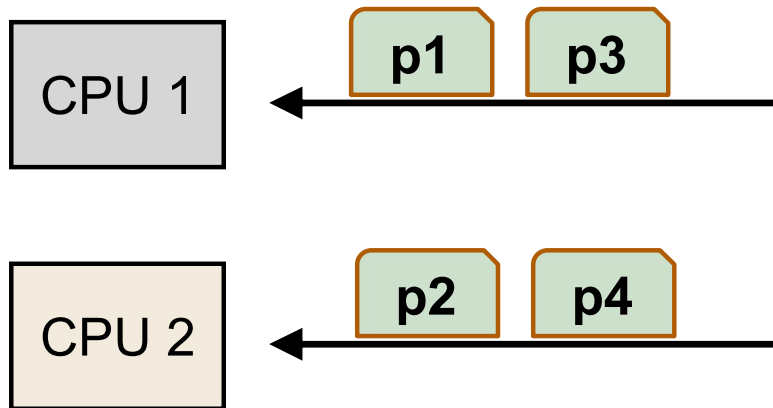


# Multitasking OS

- 1 CPU: timesliced execution of tasks



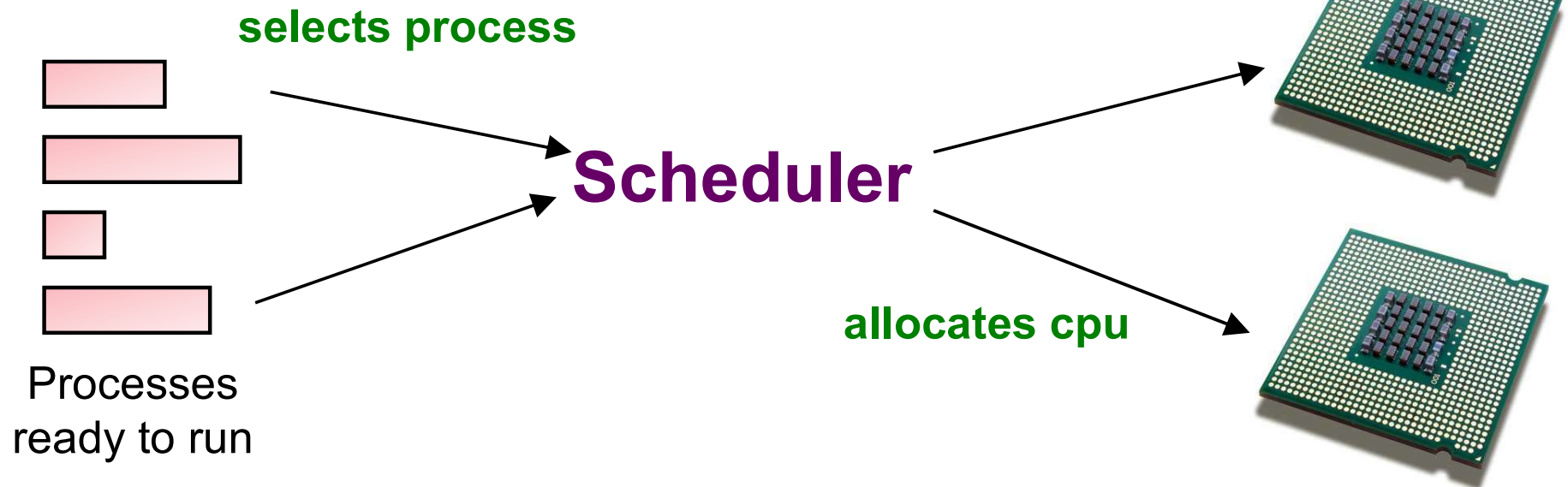
- Multiprocessor: timeslicing on  $n$  CPUs



# Scheduling in OS: A definition

- Problems with having multiple processes:
  - ❑ If [*ready-to-run processes*] > [*CPUs*], which should be chosen?
    - Similar idea in *thread-level scheduling*
  - ❑ Known as the **scheduling problem**
- Terminology:
  - ❑ **Scheduler**
    - Part of the OS that makes scheduling decision
  - ❑ **Scheduling algorithm**
    - The algorithm used by scheduler

# Scheduling: Illustration



- Each process has different requirement of CPU time
  - **Process behavior**
- Multiple ways to allocate
  - Influenced by the **process environment**
  - Known as **scheduling algorithms**
- A number of **criteria to evaluate the scheduler**

# 3 Types of Processing Environment

## 1. Batch Processing:

- ❑ No user: No interaction required, No need to be responsive
- ❑ Not considered here.

## 2. Interactive (or Multiprogramming):

- ❑ With active user interacting with system
- ❑ Should be responsive, consistent in response time

## 3. Real time processing:

- ❑ Have deadline to meet
- ❑ Usually periodic process

# Criteria for Scheduling Algorithms

- **Many criteria** to evaluate an scheduling algorithm

## Criteria for **all** processing environments:

- **Fairness:**

- Should get a fair share of CPU time
  - per process basis OR
  - per user basis
- Also mean **no starvation**

- **Balance:**

- All parts of the computing system should be utilized

# When to perform scheduling?

- Two types of scheduling policies
- **Non-preemptive (Cooperative)**
  - A process stayed scheduled until:
    1. it blocks OR
    2. give up the CPU voluntarily
- **Preemptive**
  - A process is given a fixed time quota to run
    - Possible to block or give up early
  - At the end of the time quota:
    - Another process get picked if available

# Scheduling a Process: Step-by-Step

1

- Scheduler is triggered ( **OS takes over** )

2

- **If Context switch is needed:**
  - Context of current running process is saved and placed on blocked queue / ready queue

3

- Pick a suitable process **P** to run base on scheduling algorithm

4

- Setup the context for **P**

5

- Let process **P** run

# SCHEDULING FOR INTERACTIVE SYSTEMS

# Criteria for **interactive environment**

## ■ **Response time:**

- Time between request and response by system

## ■ **Predictability:**

- Variation in response time, lesser variation == more predictable

**Preemptive** scheduling algorithms are used to ensure good response time

→ Scheduler needs to run **periodically**

# Ensuring **Periodic Scheduler**

## ■ **Questions:**

- ❑ How can the scheduler "take over" the CPU periodically?
- ❑ How can we ensure the user program can never stop the scheduler from executing?

## ■ **Ingredients for answer:**

- ❑ **Timer interrupt** = Interrupt that goes off periodically (based on hardware clock)
- ❑ OS ensure timer interrupt cannot be intercepted by any other program

➔ Timer interrupt handler **invokes scheduler**

# Terminology: **Timer** & Time Quantum

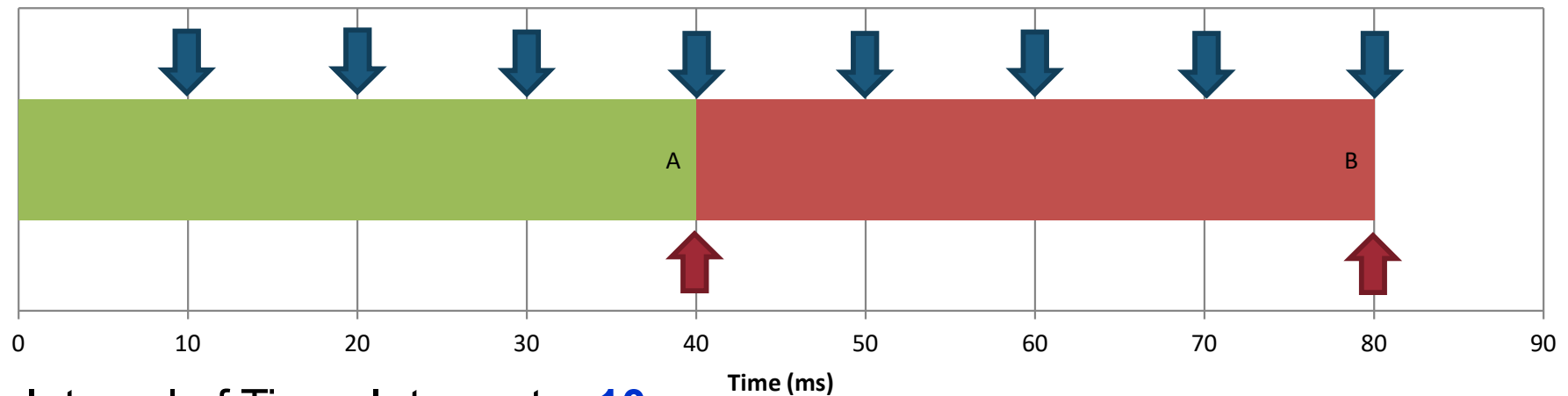
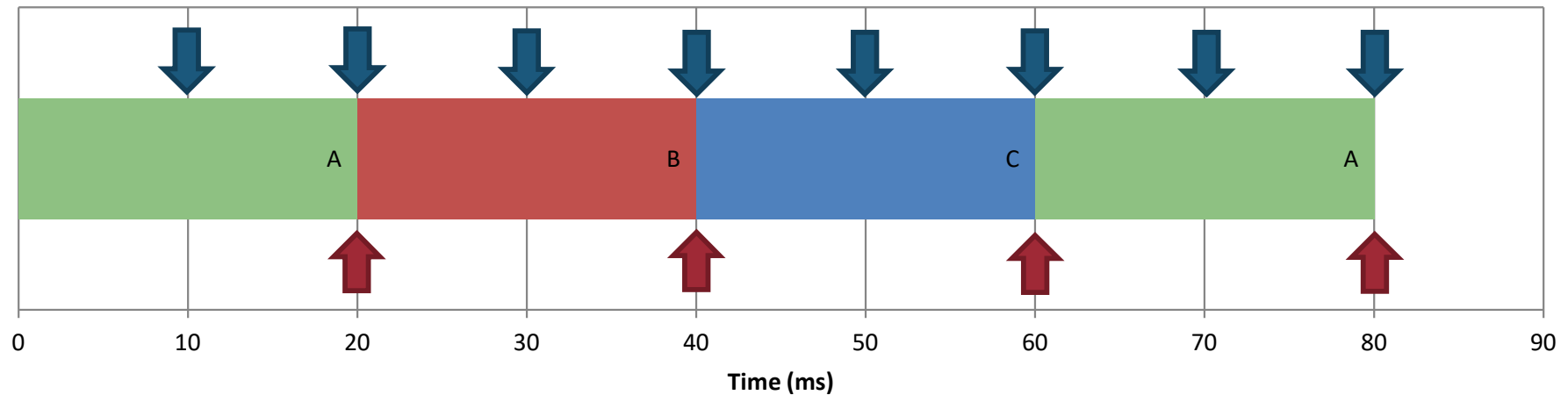
## ■ Interval of Timer Interrupt:

- ❑ OS scheduler is triggered every timer interrupt
- ❑ Typical values (**1ms to 10ms**)

## ■ Time Quantum:

- ❑ Execution duration given to a process
- ❑ Could be ***constant*** or ***variable*** among the processes
- ❑ Must be multiples of interval of timer interrupt
- ❑ Large range of values (commonly **5ms to 100ms**)

# Illustration: ITI vs Time Quantum



# Scheduling Algorithms:

- Algorithms covered:
  1. Round Robin (RR)
  2. Priority Based

# Round Robin: **RR**

## ■ General Idea:

- ❑ Tasks are stored in a **FIFO queue**
- ❑ Pick the **first task from queue** front to run until:
  - A fixed **time slice (quantum)** elapsed
  - The task gives up the CPU voluntarily
  - The task blocks
- ❑ The task is then **placed at the end of queue** to wait for another turn
  - Blocked task will be moved to other queue to wait for its request
- ❑ When blocked task is ready again, it is placed at the end of queue

# Round Robin: **RR**

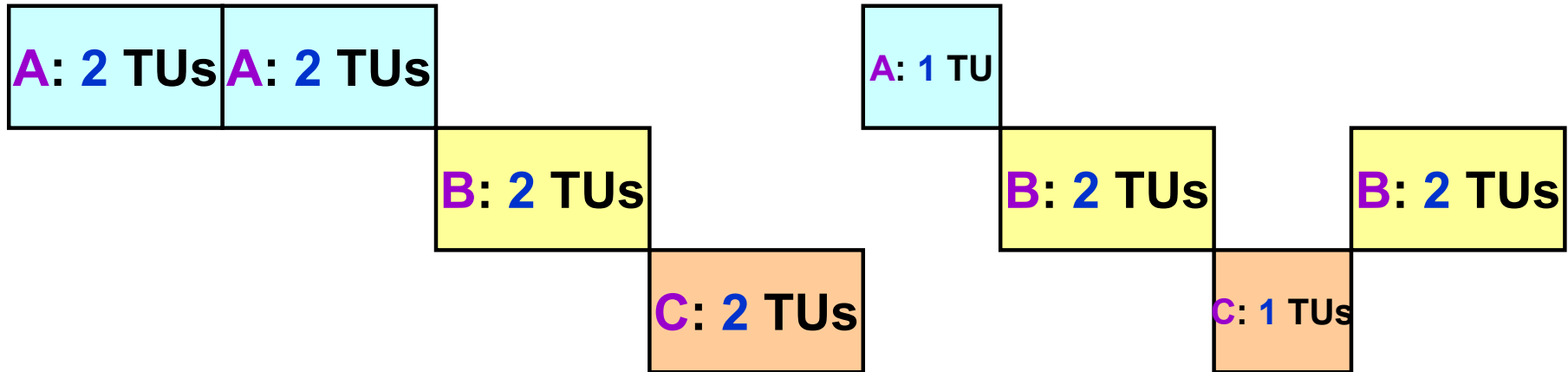
(cont)

## ■ Notes:

- ❑ Basically a preemptive version of FCFS
- ❑ **Response time guarantee:**
  - Given  $n$  tasks and quantum  $q$
  - Time before a task get CPU is bounded by  $(n-1)q$
- ❑ **Timer interrupt needed:**
  - For scheduler to check on quantum expiry
- ❑ The **choice of time quantum** duration is important:
  - **Big quantum:** Better CPU utilization but longer waiting time
  - **Small quantum:** Bigger overhead (worse CPU utilization) but shorter waiting time

# Round Robin: Illustration

Execution Timeline



# Priority Scheduling

## ■ General Idea:

- ❑ Some processes are more important than others
  - Cannot treat all process as equal
- ❑ Assign a **priority value** to all tasks
- ❑ Select task with **highest priority value**

## ■ Variants:

### ❑ **Preemptive version:**

- Higher priority process can preempts running process with lower priority

### ❑ **Non-preemptive version:**

- Late coming high priority process has to wait for next round of scheduling

# Priority Scheduling: Illustration

Tasks

Arrival Time

Priority (1=highest)

**A: 8 TUs**

Time **0**

**3**

**C: 3 TUs**

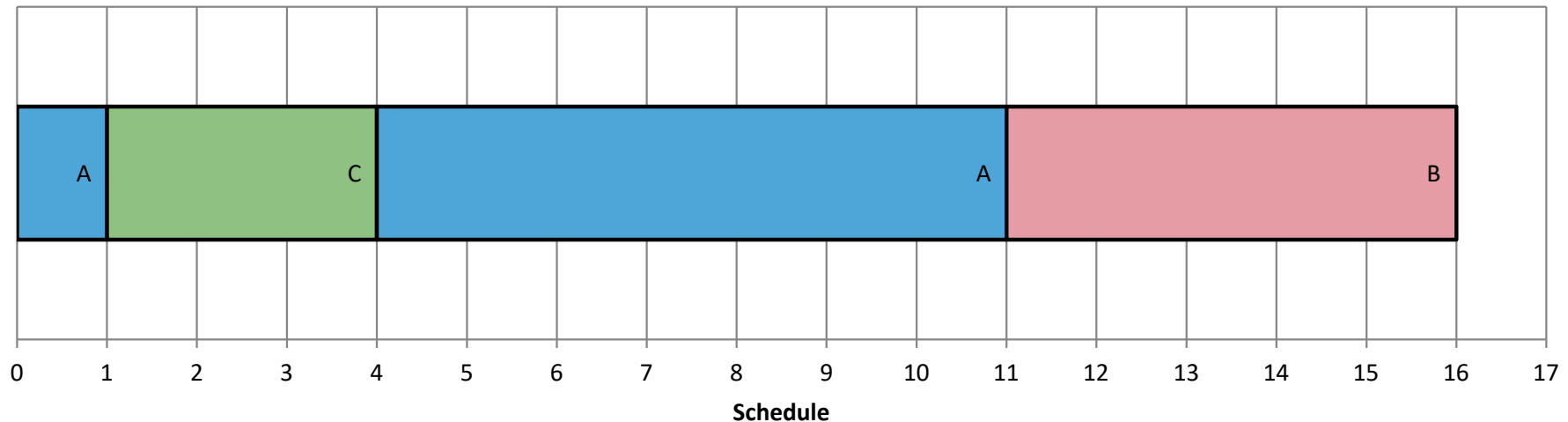
Time **1**

**1**

**B: 5 TUs**

Time **1**

**5**



# Priority Scheduling: **Shortcomings**

- Low priority process can **starve**:
  - ❑ High priority process keep hogging the CPU
  - ❑ Worse in preemptive variant
- Possible solutions:
  - ❑ Decrease the priority of currently running process after every time quantum
  - ❑ Give the current running process a time quantum
- Hard to guarantee or control the exact amount of CPU time given to a process using priority

# SCHEDULING FOR REAL-TIME SYSTEMS

# Scheduling Algorithms:

- Algorithms covered:

1. **Fixed Priority Scheduling**

2. **Rate Monotonic Scheduling (RMS)**

3. **Earliest Deadline First Scheduling (EDF)**

# Real-Time Systems: **Special Characteristics**

- Processes (Called “tasks” in RT systems) in real-time systems are often periodic:
  - ❑ GPS produces a reading 10 times per second. So we read every 0.1 seconds.
  - ❑ Serial data operates at 9600 bps or 960 cps. So we get one character every 0.001 seconds.
  - ❑ We read accelerometers at fixed intervals  $\Delta t$ , so we can integrate it to get velocity using  $v_{t+1} = v_t + a \cdot \Delta t$
- Tasks will only run for a limited time before it suspends until the next reading, calculation, etc.

# Real-Time Systems: **Special Characteristics**

- Therefore, a task  $i$  in real-time systems can be characterized by two parameters:
  - $P_i$ : The period of the task.
  - $C_i$ : The running time of the task before it is blocked again waiting for the next event.
    - Note: If a task execution time is unknown we use the worst-case execution time WCET.
- For convenience we usually work in terms of CPU cycles rather than actual time:
  - Easily convertible:

$$CPU\ Cycles = \frac{running\ time}{time\ per\ cycle}$$

# Fixed Priority Scheduling

- In Fixed Priority, each task is manually assigned a priority.
  - Higher priority tasks are picked first.
  - If a higher priority task becomes ready, it pre-empt the lower-priority task.
    - Works the same way that we saw earlier, except that tasks are periodic, and run for a fixed time before becoming blocked until the next period.
    - All tasks are ready at time  $t_0$ .
- Note: In all cases we assume that the scheduler is pre-emptive.

# Fixed Priority Scheduling

- We have the following set of 3 tasks with the periods and running times shown in CPU cycles.
  - Lower priority number = higher priority.

| Task | Pi | Ci | Priority |
|------|----|----|----------|
| T1   | 6  | 1  | 0        |
| T2   | 4  | 2  | 2        |
| T3   | 5  | 1  | 1        |

# Fixed Priority Scheduling

- We can get the schedule as shown:

| Time | Running Task | Ready Tasks                 |
|------|--------------|-----------------------------|
| 0    | T1           | T3, T2 (T1 already running) |
| 1    | T3           |                             |
| 2    | T2           |                             |
| 3    | T2           |                             |
| 4    | T2           | T2                          |
| 5    | T3           | T3                          |
| 6    | T1           | T1                          |
| 7    | T2           |                             |
| 8    | T2           | T2                          |
| 9    | T2           |                             |
| 10   | T3           | T3                          |
| 11   |              |                             |
| 12   | T1           | T1, T2                      |
| 13   | T2           |                             |
| 14   | T2           |                             |
| 15   | T3           | T3                          |
| 16   | T2           | T2                          |
| 17   | T2           |                             |
| 18   | T1           | T1, T3                      |
| 19   | T3           |                             |
| 20   | T2           | T2                          |
| 21   | T2           |                             |
| ...  | ...          | ...                         |

# Deadlines

- Deadlines are important in real-time scheduling:
  - A task must generally finish running before it is due to run again.
    - Remember: The periods correspond to when new data arrives.
    - We assume that we lose the previous data if we don't finish running before the next data comes.
      - Deadline for T2 shown below. Deadlines for T1 and T3 follow the same idea.

| Time | Running Task | Ready Tasks                 |
|------|--------------|-----------------------------|
| 0    | T1           | T3, T2 (T1 already running) |
| 1    | T3           |                             |
| 2    | T2           |                             |
| 3    | T2           |                             |
| 4    | T2           | T2                          |
| 5    | T3           | T3                          |
| 6    | T1           | T1                          |
| 7    | T2           |                             |

← Deadline for previous run of T2

# Missed Deadlines

- Consider the following task characteristics.

| Task | Pi | Ci | Priority |
|------|----|----|----------|
| T1   | 6  | 1  | 0        |
| T2   | 4  | 2  | 2        |
| T3   | 5  | 2  | 1        |

- We get the following schedule:

| Time | Running Task | Ready Tasks                 |
|------|--------------|-----------------------------|
| 0    | T1           | T3, T2 (T1 already running) |
| 1    | T3           |                             |
| 2    | T3           |                             |
| 3    | T2           |                             |
| 4    | T2           | T2                          |
| 5    |              | T3                          |

1. This is from the first run of T2

2. The run of T2 cannot complete before the next time it is due to run. Missed deadline!

# Missed Deadlines

- If we can schedule a series of tasks so that they can all meet their deadlines, we say that the tasks are “schedulable”.
  - In our earlier example, there **may** be a different ordering of priorities where all tasks meet their deadlines.
- Challenge: How do we find this optimal assignment of priorities?

# Rate Monotonic Scheduling: **RMS**

- One solution:
  - ❑ Prioritize according to period.
    - Shorter period = higher priority.
  - ❑ Rationale:
    - Tasks with shorter periods are likelier to miss their deadlines
    - Thus, we let them run first.
- This way of scheduling is called “Rate Monotonic Scheduling”
  - ❑ RMS produces the optimal assignment of priorities.

# RMS Example 1

- We consider the schedule for this set of tasks:

| Task | Pi | Ci |
|------|----|----|
| T1   | 6  | 2  |
| T2   | 8  | 2  |
| T3   | 4  | 1  |

Task priority: T3 T1 T2

| Time | Running Task | Ready Tasks         |
|------|--------------|---------------------|
| 0    | T3           | T1, T2 (T3 running) |
| 1    | T1           |                     |
| 2    | T1           |                     |
| 3    | T2           |                     |
| 4    | T3           | T3                  |
| 5    | T2           |                     |
| 6    | T1           | T1                  |
| 7    | T1           |                     |
| 8    | T3           | T3, T2              |
| 9    | T2           |                     |
| 10   | T2           |                     |
| 11   |              |                     |
| 12   | T3           | T3, T1              |
| 13   | T1           |                     |
| 14   | T1           |                     |
| 15   |              |                     |
| 16   | T3           | T3, T2              |
| 17   | T2           |                     |
| 18   | T1           | T1                  |
| 19   | T1           |                     |
| 20   | T3           | T3                  |
| 21   | T2           |                     |
| 22   |              |                     |
| 23   |              |                     |
| 24   | T3           | T3, T1, T2          |
| ...  | ...          | ...2                |

# RMS Example 2

- Since RMS is optimal, does this mean that tasks (at least those that don't overload the CPU) are always schedulable?
- Consider the case that T2 now takes 3 cycles instead of 2:

| Task | Pi | Ci |
|------|----|----|
| T1   | 6  | 2  |
| T2   | 8  | 3  |
| T3   | 4  | 1  |

Task priority: T3 T1 T2

| Time | Running Task | Ready Tasks         |
|------|--------------|---------------------|
| 0    | T3           | T1, T2 (T3 running) |
| 1    | T1           |                     |
| 2    | T1           |                     |
| 3    | T2           |                     |
| 4    | T3           | T3                  |
| 5    | T2           |                     |
| 6    | T1           | T1                  |
| 7    | T1           |                     |
| 8    | T2           | T3, T2              |

Missed Deadline

# Testing Schedulability

- Most obvious way: “unroll” the schedule until it repeats.
  - The schedule will repeat after the “hyper-period”, which is the lowest-common-multiple of all the periods.
  - In our example:
    - $\text{LCM}(4, 6, 8) = 24$
    - So, our unrolling is complete. First example is schedulable.
- Tedious, and sometimes impractical.
  - E.g.  $\text{LCM}(3, 7, 11) = 231$  – Need to compute 231 rows!

| Task | Pi | Ci |
|------|----|----|
| T1   | 6  | 2  |
| T2   | 8  | 2  |
| T3   | 4  | 1  |

# CPU Utilization

- CPU Utilization is the percentage of time that the CPU is being used.
- For periodic processes, this is given by:

$$\sum_i \frac{C_i}{P_i}$$

- CPU utilization of our two RMS examples:
  - Example 1 =  $2/6 + 2/8 + 1/4 = 0.83$
  - Example 2 =  $2/6 + 3/8 + 1/4 = 0.96$
- In all scheduling algorithms, it is necessary that  $U \leq 1.0$

# Utilization Bound

- We can test RMS schedulability by using the Liu and Leyland bound:

- A set of tasks is schedulable under RMS if:

$$U \leq n \left( 2^{\frac{1}{n}} - 1 \right)$$

- In our example,  $n = 3$ , so we need:

$$U \leq 3 \left( 2^{\frac{1}{3}} - 1 \right) = 0.78$$

- Note how RMS example 1  $U = 0.83 > 0.78$ , but it is schedulable.
  - Liu and Leyland is a sufficient but not necessary condition.

# Utilization Bound

- Common Liu and Leyland bounds:

|              |              |                   |
|--------------|--------------|-------------------|
| $B(1)=1.0$   | $B(4)=0.756$ | $B(7)=0.728$      |
| $B(2)=0.828$ | $B(5)=0.743$ | $B(8)=0.724$      |
| $B(3)=0.779$ | $B(6)=0.734$ | $U(\infty)=0.693$ |

- If  $U$  is less than or equal to the bound for the number of tasks, we do not need further testing.
  - However, if  $U$  is larger, the tasks may still be schedulable and we need to do further testing.

# Critical Instance Analysis

1. Sort  $T$  by period of each task, if  $T$  is not already sorted.  
We will assume that  $T_1$  has the shortest period,  $T_2$  has the 2<sup>nd</sup> shortest, etc.

2. For each task  $T_i \in T$ , recursively compute  $S_{i0}, S_{i1}, \dots$  where:

$$\text{a. } S_{i,0} = \sum_{j=1}^i C_j$$

$$\text{b. } S_{i,(x+1)} = C_i + \sum_{j=1}^{i-1} C_j \times \left\lceil \frac{S_{i,x}}{P_j} \right\rceil$$

Stop when  $S_{i,(x+1)} = S_{i,x}$ . Call this  $S_{i,(x+1)}$  the final value  $S_{i,F}$ . I.e. let  $S_{i,F} = S_{i,(x+1)}$  when the iteration terminates.

3. If  $S_{i,F} < D_i$ , then the task  $i$  is schedulable and will not miss its deadlines.

# Critical Instance Analysis

## ■ RMS Example 1:

$$S_{3,0} = 1$$

$$S_{3,1} = 1 < 4$$

## ■ T3 is schedulable.

$$S_{1,0} = C_3 + C_1 = 1 + 2 = 3$$

$$\begin{aligned} S_{1,1} &= 2 + \left\lceil \frac{3}{4} \right\rceil \times 1 \\ &= 3 < 6 \end{aligned}$$

## ■ T1 is schedulable.

| Task | Pi | Ci |
|------|----|----|
| T1   | 6  | 2  |
| T2   | 8  | 2  |
| T3   | 4  | 1  |

# Critical Instance Analysis

## ■ RMS Example 1:

$$S_{2,0} = 1 + 2 + 2 = 5$$

$$\begin{aligned} S_{2,1} &= 2 + \left\lceil \frac{5}{4} \right\rceil \times 1 + \left\lceil \frac{5}{6} \right\rceil \times 2 \\ &= 2 + 2 + 2 = 6 \end{aligned}$$

$$\begin{aligned} S_{2,2} &= 2 + \left\lceil \frac{6}{4} \right\rceil \times 1 + \left\lceil \frac{6}{6} \right\rceil \times 2 \\ &= 2 + 2 + 2 = 6 \end{aligned}$$

$$6 < 8$$

| Task | Pi | Ci |
|------|----|----|
| T1   | 6  | 2  |
| T2   | 8  | 2  |
| T3   | 4  | 1  |

- T2 is schedulable.
- Since T3, T1 and T2 are schedulable, all will meet their deadlines.

# Critical Instance Analysis

## ■ RMS Example 2

- T3 and T1 are the same as before, so we will only work on T2.

$$S_{2,0} = 1 + 2 + 3 = 6$$

$$S_{2,1} = 3 + \left\lceil \frac{6}{4} \right\rceil \times 1 + \left\lceil \frac{6}{6} \right\rceil \times 2 = 3 + 2 + 2 = 7$$

$$S_{2,2} = 3 + \left\lceil \frac{7}{4} \right\rceil \times 1 + \left\lceil \frac{7}{6} \right\rceil \times 2 = 3 + 2 + 4 = 9 > 8$$

| Task | Pi | Ci |
|------|----|----|
| T1   | 6  | 2  |
| T2   | 8  | 3  |
| T3   | 4  | 1  |

- We see that T2 is not schedulable.

# Earliest Deadline First

- RMS is a fixed priority scheme:
  - ❑ Issue: Even as we approach the deadline of a task, it can be pre-empted by higher priority tasks.
  - ❑ What if we change the schedule so that as a task approaches its deadline, its priority is upgraded?
- This is Earliest Deadline First (EDF) scheduling.

# EDF Example

- Let's consider RMS Example 2, but scheduled it under EDF instead.

| Task | Pi | Ci |
|------|----|----|
| T1   | 6  | 2  |
| T2   | 8  | 3  |
| T3   | 4  | 1  |

| Time | Running | Deadline   | Ready Queue |
|------|---------|------------|-------------|
| 0    | T3      | T1, T2, T3 | T1, T2      |
| 1    | T1      |            | T2          |
| 2    | T1      |            | T2          |
| 3    | T2      |            |             |
| 4    | T2      | T3         | T3          |
| 5    | T2      |            |             |
| 6    | T3      | T1         | T1          |
| 7    | T1      |            |             |
| 8    | T1      | T2, T3     | T3, T2      |
| 9    | T3      |            |             |
| 10   | T2      |            |             |
| 11   | T2      |            |             |
| 12   | T2      | T1, T3     | T3, T1      |
| 13   | T3      |            | T1          |
| 14   | T1      |            |             |
| 15   | T1      |            |             |
| 16   | T3      | T2, T3     | T2          |
| 17   | T2      |            |             |
| 18   | T2      | T1         | T1          |
| 19   | T2      |            | T1          |
| 20   | T1      | T3         | T3          |
| 21   | T1      |            | T3          |
| 22   | T3      |            |             |
| 23   |         |            |             |
| 24   |         | T1, T2, T3 |             |

# Summary

## ■ Scheduling in OS:

- ❑ Basic definition
- ❑ Factors that affect scheduling
  - Process, Environment
- ❑ Criteria of good scheduling

## ■ Scheduling Algorithms:

- ❑ RR, Priority scheduling for Interactive System
- ❑ Priority, RMS and EDF scheduling for Real Time Systems.

# Reference

- Modern Operating System (3<sup>rd</sup> Edition)
  - ❑ Chapter 2.4
- Operating System Concepts (7<sup>th</sup> Edition)
  - ❑ Chapter 5
- Operating Systems: Three Easy Pieces
  - ❑ By Arpaci-Dusseau and Arpaci-Dusseau
  - ❑ <http://pages.cs.wisc.edu/~remzi/OSTEP/>